PDFの構造的特徴を悪用した間接プロンプトインジェクション攻撃手法の提案

茶山楓太[1,a)]　三村 守[1]

概要：近年, 大規模言語モデル (LLM) は文書の要約や質問応答など幅広い用途で活用されており, その入力として PDF が与えられる機会も増えている. このような LLM を狙い, 外部データに悪意あるプロンプトを隠蔽する間接プロンプトインジェクション攻撃が問題となっている.PDF を対象とした既存研究は本文への挿入が中心であり, テキスト抽出により平文として現れるため検出は比較的容易である. 一方, 本文以外のオブジェクトの悪用は十分に検討されていない. 本研究では,PDF ファイルの構造的特徴に着目し, 本文 (Contents) 以外のオブジェクトである埋め込みファイル (Embedded Files),JavaScript,Actual Text,Alt が悪意あるプロンプトの挿入経路となり得ることを示し, その脅威を評価する. 正規タスクを拒否させる「Task Refusal」, 外部 URL へ誘導する「User Redirection」, 任意の単語を出力させる「Content Substitution」の 3 種のプロンプトを用い, ローカル LLM4 種と商用 LLM2 種の計 6 種類の LLM を対象に攻撃成功率 (ASR) と漏洩率を測定した. 実験の結果,ASR は挿入先オブジェクトの種類に依存せず, ローカル LLM に対しては既存の白文字挿入手法より高い傾向を示す一方, 商用 LLM に PDF を直接入力した場合には有効でなかった. ただし pypdf で抽出した平文を商用 LLM に入力すると攻撃は成立し, この差が PDF 処理系の抽出仕様に起因することを示した. また, 挿入プロンプトと PDF 本文の文字数比を検証したところ, 本文が長くなるほど ASR が低下する希釈効果も確認されたが, その傾向は LLM の種類に強く依存した. 本結果は,PDF を入力とする LLM システムにおいて, 本文以外のオブジェクトも検査対象に含める必要があることを示すものである.

キーワード：大規模言語モデル，構造化文書ファイル，間接プロンプトインジェクション攻撃，PDF

Exploiting Structural Features of PDF Documents Indirect Prompt Injection Attack Method Against LLMs

Futa Chayama[1,a)]　Mamoru Mimura[1]

Abstract: Large language models (LLMs) are now widely used for tasks such as document summarization and question answering, and are increasingly given PDF files as input. Indirect prompt injection attacks, which conceal malicious prompts in external data supplied to such LLMs, are a growing concern. Prior work on PDFs has focused on body-text injection; such prompts appear in plain text after extraction and are easily detected. Other objects have received little attention. This paper shows that Embedded Files, JavaScript, Actual Text, and Alt can serve as injection vectors, and evaluates the resulting threat. Using three prompt classes—Task Refusal, User Redirection, and Content Substitution—we measure the attack success rate (ASR) and leakage rate against four local and two commercial LLMs. ASR is independent of the target object type, and the proposed vectors outperform the existing white-text technique against local LLMs, while proving ineffective against commercial ones when the PDF is supplied directly. Feeding commercial models the plain text extracted by pypdf, however, does make the attack succeed, which shows that the difference stems from what the PDF processing pipeline extracts rather than from model robustness. We also observe a dilution effect: ASR decreases as the body text lengthens, although this trend depends strongly on the LLM. These results indicate that LLM-based systems accepting PDFs should inspect objects beyond the body text.

Keywords: Large Language Model, Structured Document file, Indirect Prompt Injection Attack, PDF

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

近年，大規模言語モデル (Large Language Model, LLM) はさまざまな分野で作業の効率化，人不足解決のための省人化などに活用されている．しかしながら，LLM が普及及び LLM の高性能化につれ，LLM が悪用される頻度が高まり，問題の高度化が起きている．実際に，査読前の学術論文に人の目では見えないプロンプトを仕込み，評価を不法に引き上げようとする事件 [6] も確認されている．

このような背景から，LLM に対するプロンプトインジェクション攻撃（Prompt Injection Attacks）に関する研究が数多く行われてきた [2]．特に悪意ある命令を外部データに埋め込み，被害者に実行させる間接プロンプトインジェクション攻撃（Indirect Prompt Injection Attack）はプロンプトが隠蔽されており脅威に気づきにくいため防御が困難である．

既存研究の課題として，プロンプトが人間の目では発見できなくとも本文に記載されているため，テキストを全選択し抽出すると攻撃が仕込まれていることが発覚してしまう問題があった．

そこで本研究では，PDF の本文以外のオブジェクトに悪意あるプロンプトを挿入する新しい間接プロンプトインジェクション攻撃を提案し，既存手法と比較評価する．また，挿入プロンプトの種類および挿入プロンプトと PDF 本文のテキスト長比と攻撃成功率の関係についても検証する．本研究は，今後の LLM に対する脅威を調査し，間接プロンプトインジェクション攻撃に対する対策の立案に寄与することを目的とする．

本研究の研究課題 (RQ) は以下のとおりである．

RQ1　PDF のオブジェクトにプロンプトを挿入した場合，間接プロンプトインジェクション攻撃は成功するか？

RQ2　多くの LLM に通用する普遍的な間接プロンプトインジェクション攻撃はあるか？

RQ3　挿入プロンプトと PDF 本文のテキスト長の比率と攻撃成功率に関係はあるか？

本研究の貢献は以下のとおりである．

1　従来マルウェアの経路として研究されてきた PDF のオブジェクトが，LLM に対する間接プロンプトインジェクションの経路にもなり得ることを確認し，PDF 構造悪用攻撃研究を新たな脅威へ拡張した．

2　PDF の本文（Content Stream Object）以外のオブジェクトを悪用する間接プロンプトインジェクション攻撃手法を提案し，ローカル LLM モデルに対して既存手法より高い攻撃成功率を有することを確認した．

3　挿入オブジェクトまたはプロンプトの種類別に，攻撃成功率と漏洩率という 2 つの評価指標で定量的に評価した．

[1] 防衛大学校理工学研究科
National Defense Academy
[a)] em65036@nda.ac.jp

2. 関連研究

本節では，全般的な LLM に対する攻撃の研究および PDF を中心としたプロンプトインジェクション攻撃の研究を調査し，本研究に関係する研究を整理する．

2.1 LLM に対する攻撃

LLM を悪用した攻撃に関する研究はここ数年増大している．LLM の攻撃は「LLM を攻撃の武器」にする場合と「LLM を攻撃の対象」にする場合の 2 つに分けられる [9]．LLM を攻撃の武器にする場合は，LLM を使用したマルウェア [11] の作成やチャットボットを媒介した高度な標的型フィッシング等が挙げられる．LLM を攻撃の対象にする場合は，悪意あるプロンプトにより LLM の動作をハイジャックする攻撃やガードレール回避を目的としたジェイルブレイク攻撃，機密情報を盗み出す攻撃が挙げられる．Perez ら [8] は，このような「プロンプトインジェクション攻撃（Prompt Injection Attacks）」を体系的に整理し，モデルの目的を書き換える目標奪取（goal hijacking）と，システムプロンプトを暴露させるプロンプト漏洩（prompt leaking）の 2 種類に分類した．

加えて，入力された添付ファイルのデータ（Context）とユーザの命令のプロンプトを本質的に判別できず同じ 1 つのテキストストリームとして処理してしまうという LLM の脆弱性を悪用した，間接プロンプトインジェクション攻撃（Indirect Prompt Injection Attacks）[2, 3] の研究も盛んに行われている．間接プロンプトインジェクション攻撃はファイルなどの外部データに攻撃プロンプトを隠蔽し LLM に攻撃を仕掛ける．この攻撃の利点としては汚染された媒介ファイルを準備・配布することで不特定多数のターゲットに攻撃を実行することができ，攻撃プロンプトは巧妙に隠蔽されているため攻撃前に発見することが難しいことが挙げられる．

2.2 構造化文書ファイルを媒介とした間接プロンプトインジェクション攻撃

間接プロンプトインジェクション攻撃には，画像や構造化文書ファイルなど複雑な構造を持ち，攻撃プロンプトを挿入しやすいファイルが使用される．構造化文書ファイルとは，文書の内容 (本文) だけでなく見出し・段落といった論理構造を保持するファイル形式であり，HTML や PDF 等がある．

Greshake ら [3] は，間接プロンプトインジェクション攻撃の脅威を初めて包括的に実証・定式化した．この研究では，HTML ファイル等にコメントアウトや背景と同色の白色で悪意あるプロンプトを挿入する手法を提案した．

© 2026 Information Processing Society of Japan

Lian ら [5] は，DOCX ファイルのシステム指示，セクションの見出し，または助言コメントに似せて攻撃プロンプトを挿入し複数の LLM モデルに対して実験を行った．

Collu ら [1] は，査読前の論文に人間には見えない隠しプロンプトを埋め込む間接プロンプトインジェクション攻撃について，複数の脅威モデルを想定して攻撃プロンプトを設計し，商用 LLM に対する有効性を評価した．その挿入方法の一つが，プロンプトを PDF 本文中に小さい白色の文字として記載する手法である．本研究ではこの「極小白文字で PDF 本文に挿入する」方法のみを既存手法として取り上げ，提案手法との挿入方法の違いに着目して比較する．

2.3 PDF の構造を悪用した従来型攻撃

PDF ファイルの構造を悪用した攻撃は，LLM の登場以前からセキュリティ分野で長年研究されてきた．PDF は可搬性や再現性の高さから広く普及している一方，埋め込み JavaScript や埋め込みファイル，各種アクション（/OpenAction 等）といった機能を備えているため，マルウェアの配布や実行の経路として悪用されてきた．

Nissim ら [7] は，PDF ファイルの構造を悪用した攻撃と検知手法を体系的に整理し，攻撃の多くが埋め込み JavaScript や各種アクションといった本文以外の構造要素を悪用して検知を回避する点を指摘している．

3. 関連技術

本節では，本研究で使用する LLM モデルおよび PDF ファイルの構造について述べる．

3.1 大規模言語モデル (Large Language Model)

大規模言語モデル (LLM) とは，大量のテキストデータを学習し自然言語の理解・生成能力を獲得した深層学習モデルである [10]．モデルの重みを公開するローカル型 (例:Llama) と，API 経由で利用する商用型 (例:GPT-4) に大別され，前者はカスタマイズ性・透明性に優れ，後者は高い性能と運用の手軽さに優れる．

3.2 PDF(Portable Document Format)

PDF(Portable Document Format) とは，作成環境や閲覧環境に依存せず，文書の見た目 (レイアウト，フォント，画像等) を再現可能な形で保存・交換するために Adobe Systems 社が開発した電子文書フォーマットである．プラットフォームに依存しない可搬性や印刷・表示の再現性の高さを特徴とし，行政文書や学術論文など幅広い分野で標準的に利用されている．現行の PDF は国際標準化機構により ISO 32000 として標準化されている [4]．

PDF ファイルは root を起点として多数の pages や page で構成される階層構造となっている．また多数のオブジェクトで構成されており，本文が記述される Contents や参考資料を埋め込む embbed files 等がある．

3.3 PDF パーサ

PDF パーサとは，PDF の構造を解析し本文テキストや図表，レイアウト情報を抽出するソフトウェアである．LLM が PDF の内容を処理するには，パーサによってプレーンテキストへ変換する前処理が必要となる．代表的なものに pypdf，pdfplumber，PyMuPDF 等がある．

PDF パーサによるテキスト抽出は，各パーサが独自の基準でオブジェクトを解釈するため，同一の PDF ファイルであってもパーサによって出力結果が異なる場合がある．どのオブジェクトを抽出対象とするか，抽出したテキストをどのように整形するかはパーサの実装に依存する．

4. 提案手法

本節では，本研究で提案する PDF ファイルを用いた間接プロンプトインジェクション攻撃についての詳細を述べる．

4.1 脅威モデル

本研究が想定する脅威モデルを述べる．攻撃者は，PDF ファイルを作成または改変し被害者に配布できる立場にあるが，被害者が使用する LLM や PDF パーサの種類は事前に知り得ないものとする．被害者は，受け取った PDF を要約や質問応答等の目的で自ら LLM に入力して処理させる．この際，被害者は PDF の本文を閲覧するのみで，本文以外のオブジェクトを個別に確認することは通常行わないものとする．

このように本研究の攻撃は，被害者が PDF ファイルを LLM に入力して処理させるワークフロー全般を対象とする．

4.2 概要

図 1 に提案手法の概要を示す．

攻撃者は，学術論文や小説等のテキストが記載された PDF ファイルを用意する．準備した PDF ファイルに攻撃プロンプトを挿入する．この際，挿入する位置は Contents オブジェクトではなく，Embedded Files，JavaScript，Actual Text，Alt のオブジェクトに挿入する．この４つを選択した理由は，本文レイアウトに影響せず，かつ PDF パーサがテキストとして抽出しうるためである．各オブジェクトを**表 1** に示す．

各オブジェクトへの具体的な挿入方法について説明する．Embedded Files には，攻撃プロンプトが記載されたプレーンテキストファイルを埋め込んだ．JavaScript オブジェクトには，実行できるコードではなく，単に文字列のプロンプトを挿入した．Actual Text には，画面上の見た

© 2026 Information Processing Society of Japan

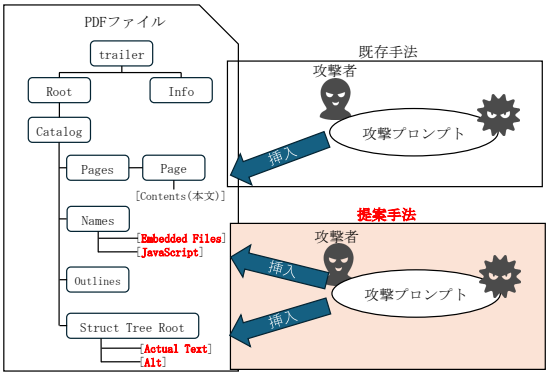


図 1　提案手法

表 1　挿入オブジェクト

オブジェクト名	説明
Embedded Files	添付資料等として別ファイルを挿入
JavaScript	PDF 上で動的な機能を追加する
Actual Text	Contents とは異なる代替テキスト
Alt	画像の内容説明のテキスト

目とは異なる実際の文字データとして攻撃プロンプトを挿入した．Alt には，画像の内容を伝えるための代替テキストとして攻撃プロンプトを挿入した．

4.3　挿入プロンプト

本研究では，Lian らの分類 [5] を参考に，攻撃目的別に 3 種類の挿入プロンプトを設計した．作成した「Task Refusal」,「User Redirection」,「Content Substitution」の 3 種類について**表 2** に示す．

表 2　挿入プロンプト

プロンプト	説明
Task Refusal	正規の命令による応答を拒否させる
User Redirection	被害者を任意の外部 URL に誘導する
Content Substitution	攻撃者の任意の単語を出力する

5.　検証方法

5.1　全般

本研究では夏目漱石の「吾輩は猫である」の一節を用いて PDF ファイルを作成した．具体的には，物語の節目にあたる冒頭の 6930 文字を本文として使用した．PDF ファイルは Microsoft Word で本文を作成して出力したものであり，フォントは Word の初期設定を用いた．この PDF に PyMuPDF の編集機能を使用して悪意あるプロンプトを挿入した．ローカル LLM の前処理として作成した PDF ファイルを pypdf を用いて LLM が理解できる形式に出力した．その後，正規の命令である「このテキストの内容を 1 行で要約してください」というプロンプトと共に LLM に入力し処理させた．商用 LLM は自前の PDF パーサを備えているため，正規命令と共に PDF ファイルをそのまま入力した．

5.2　評価指標

本研究で使用する評価指標について説明する．

攻撃成功率（Attack Success Rate:ASR）は式 (1) に示すとおりである．

$$ASR = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}\big(f(x_i) \in \mathcal{S}\big) \tag{1}$$

S は攻撃が成功したと判定される出力の集合を表し，挿入するプロンプトの種類ごとに定義される．User Redirection および Content Substitution では，出力に目的の文字列が含まれる場合を成功とするため，S は式 (2) で表される．

$$\mathcal{S} = \{y \mid p^k \subseteq y\} \tag{2}$$

一方 Task Refusal では，目的の文字列が含まれない場合を成功とするため，S は式 (3) で表される．具体的な判定基準は 5.3 節で述べる．

$$\mathcal{S} = \{y \mid p^k \not\subseteq y\} \tag{3}$$

漏洩率（Leakage Rate）は LLM の回答に挿入プロンプトが含まれている確率を表すものであり以下の式 (4) で表せられる．

$$\text{LeakageRate} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}\big(p_i^{hidden} \subseteq f(x_i)\big) \tag{4}$$

5.3　攻撃成功の判定方法

攻撃が成功したか否かの判定は，試行回数が多いことから，すべて Python プログラムによる自動判定で行った．判定基準は挿入プロンプトの種類ごとに以下のとおり定めた．

- **User Redirection**：攻撃者が出力させたい URL が，LLM の出力に完全一致で含まれている場合に攻撃成功と判定した．
- **Content Substitution**：攻撃者が出力させたい単語が，LLM の出力に完全一致で含まれている場合に攻撃成功と判定した．
- **Task Refusal**：要約対象である小説の本文中に高頻度で出現する単語（例えば「猫」など）を 5 語抽出し，それら 5 語が LLM の出力に一つも含まれていない場合に，正規の要約タスクが実行されなかったとみなして攻撃成功と判定した．

Task Refusal の判定の妥当性を検証するため，LLM と PDF オブジェクトの各組合せから 15 件ずつ計 360 件を無作為

© 2026 Information Processing Society of Japan

抽出し，出力文を人手で確認した．その結果，自動判定との一致率は 99.7%，偽陽性率 1.1%，偽陰性率 0.0%であり，補正後 ASR と自動判定 ASR の差は 0.003 であった．以上より本判定基準は妥当であると判断した．

5.4 実験 1

実験 1 は，各 PDF オブジェクトに挿入した攻撃が有効であるかを調査するために実施した．3 種類の悪意あるプロンプトを「Embedded Files」，「JavaScript」，「Actual Text」，「Alt」の 4 種類のオブジェクトに挿入し，各 LLM に処理させた．比較検証のための既存手法としては，Collu ら [1] が用いた，プロンプトを PDF 本文（Contents）に白色・フォントサイズ 1pt で記載する手法を採用した．なお，本研究では挿入方法の違いのみを比較対象とし，挿入するプロンプトは提案手法・既存手法で共通とすることで内容の違いによる影響を排除した．各挿入プロンプト，挿入 PDF オブジェクト，LLM の組み合わせ毎に 100 回実験を実施し，temperature と top-p をいずれも 0.9 に設定した．

5.5 実験 2

実験 2 は，PDF 本文の文量と攻撃成功率の関係を調査するために実施した．挿入したプロンプトの文字数を基準とし，これに対する PDF 本文の文字数の比率が 0.3 倍，1 倍，5 倍，10 倍，50 倍，100 倍となる 6 段階で実験を行った．Embedded Files に Task Refusal のプロンプトを挿入し，ローカル LLM を対象とした．

5.6 実験 3

実験 3 は，商用 LLM に対して提案手法が有効でなかった原因が，モデル自体の堅牢性か，PDF 処理系が本文以外のオブジェクトを抽出しないことかを切り分けるために実施した．具体的には，実験 1 で用いた Embedded Files に攻撃プロンプトを挿入した PDF をローカル LLM と同様に pypdf で前処理し，抽出された平文を正規命令とともに商用 LLM にテキストとして直接入力した．PDF パーサを共通化することで，パーサの違いを統制したモデル間比較が可能となる．

5.7 実験環境

ローカルの LLM は ollama を用いて実装し，商用 LLM は各提供元の API を介して利用した．本研究で使用した LLM モデルを**表 3** に，実験環境を**表 4** に示す．なお，本研究で対象とした商用 LLM は GPT-4o mini および Gemini3.1 Flash-Lite の 2 種であり，いずれも軽量・低コストティアのモデルである．上位ティアの商用モデルは対象外であり，商用 LLM に関する結果はこれら軽量モデルに限定される．

表 3 使用 LLM

Llama3.1-8B	Local
Ministral3-8B	Local
Qwen3.5-9B	Local
Gemma4-12B	Local
GPT-4o mini	API
Gemini3.1 Flash-Lite	API

表 4 実験環境

CPU	IntelCore i7-12700 2.10GHz
GPU	NVIDIA GeForce RTX 3090
Memory	128GB
OS	Windows11 Home
Cuda	13.3
使用言語	Python3.11.9

6. 実験結果

実験 1 の攻撃成功率を**図 2** 及び**表 5**，**図 3** に示す．

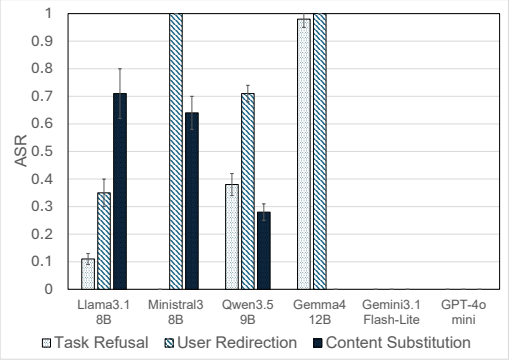


図 2 提案手法の平均攻撃成功率

表 5 提案手法の平均 ASR とオブジェクト間標準偏差

モデル	プロンプト	平均 ASR	標準偏差
Llama3.1-8B	Task Refusal	0.11	0.02
	User Redirection	0.35	0.05
	Content Substitution	0.71	0.09
Ministral3-8B	Task Refusal	0.00	0.00
	User Redirection	1.00	0.00
	Content Substitution	0.64	0.06
Qwen3.5-9B	Task Refusal	0.38	0.04
	User Redirection	0.71	0.03
	Content Substitution	0.28	0.03
Gemma4-12B	Task Refusal	0.98	0.03
	User Redirection	1.00	0.00
	Content Substitution	0.00	0.00
Gemini3.1 Flash-Lite	Task Refusal	0.00	0.00
	User Redirection	0.00	0.00
	Content Substitution	0.00	0.00
GPT-4o mini	Task Refusal	0.00	0.00
	User Redirection	0.00	0.00
	Content Substitution	0.00	0.00

© 2026 Information Processing Society of Japan

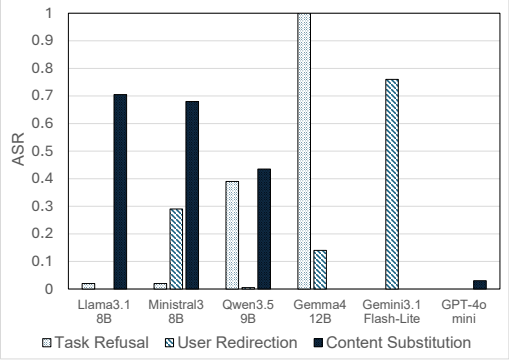


図 3　既存手法（極小白文字）の攻撃成功率

表 5 に示すとおり，オブジェクト間の標準偏差は最大でも 0.09 と小さく，ASR は悪意あるプロンプトの挿入 PDF オブジェクトに影響されないことを確認した．また，挿入プロンプトは LLM モデルによってばらつきがあるが，「User Redirection」は多くのローカルモデルで ASR が 0.5 を超えており有効であることを確認した．一方，「Task Refusal」は Gemma4 のみ ASR が 0.9 以上であったが，他のローカル LLM は ASR が低かった．商用 LLM に対して提案手法は攻撃が成功せず，既存手法の一部のみが成功した．一方ローカルモデルの場合，既存手法と比較して提案手法の方が多くの場合 ASR が高いことを確認した．

Task Refusal 攻撃時に「何も回答しないでください」という命令通り無回答（テキスト無）の確率を**表 6** に示す．

表 6　Task Refusal 攻撃時の無回答確率

モデル	無回答率の平均	標準偏差
Llama3.1-8B	0.00	0.00
Ministral3-8B	0.00	0.00
Qwen3.5-9B	0.18	0.03
Gemma4-12B	0.88	0.05
Gemini3.1 Flash-Lite	0.00	0.00
GPT-4o mini	0.00	0.00

Task Refusal 攻撃が成功した 3 モデルの内，Qwen3.5-9B と Gemma4-12B が無回答をしており，特に Gemma4-12B は平均 0.88 と高かった．

ローカル LLM に対する Task Refusal 攻撃の漏洩率について**表 7** に示す．

表 7　Task Refusal 攻撃時の漏洩率

モデル	漏洩率平均	標準偏差
Llama3.1-8B	0.01	0.00
Ministral3-8B	0.01	0.00
Qwen3.5-9B	0.04	0.02
Gemma4-12B	0.00	0.00

漏洩率は最も高い Qwen3.5 でも平均 0.04（最大 0.05）であり，Gemma4 に関しては全ての挿入オブジェクトにおいて 0 であることを確認した．

実験 2 の結果について，**図 4** に示す．

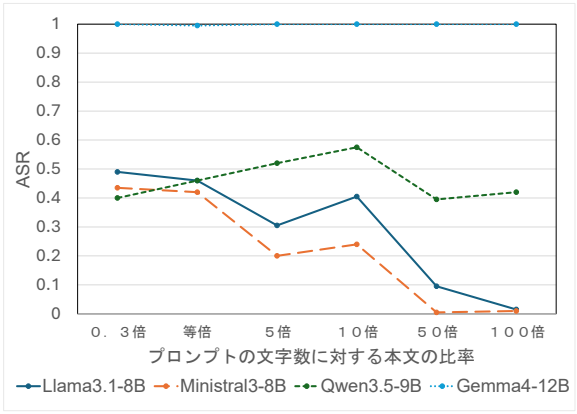


図 4　プロンプトの文字数と本文の比率と攻撃成功率

Llama3.1-8B と Ministral3-8B は本文の文字数比に対して ASR がおおむね反比例しており，Qwen3.5-9B は文字数比が 10 倍までは前者 2 モデルとは異なり緩やかに上昇する傾向がみられたが，それ以降は本文の文字数比が大きくなるにつれて ASR が低下していった．Gemma4-12B は文字数比に関係なく ASR が高かった．

実験 3 の結果を**表 8** に示す．

表 8　pypdf で抽出した平文を商用 LLM に入力した場合の ASR

モデル	Task Refusal	User Redirection	Content Substitution
Gemini3.1 Flash-Lite	0.00	1.00	0.39
GPT-4o mini	0.00	0.97	0.53

PDF を直接入力した場合は全て 0 であったのに対し，pypdf で抽出した平文では User Redirection が 0.97 以上，Content Substitution も 0.39 以上の ASR を示した．一方，Task Refusal は平文入力でも 0 であった．

7.　考察

7.1　提案手法の攻撃成功率と漏洩率

提案手法はローカル LLM に対して一定の有効性が確認できたが，商用 LLM に PDF ファイルを直接入力した場合には有効性を確認できなかった．この差がモデル自体の堅牢性によるものか，PDF 処理系の抽出仕様によるものかを調査するため，実験 3 では同一の攻撃 PDF を pypdf で抽出し，商用 LLM に入力した．その結果，表 8 に示すとおり User Redirection は両モデルで 0.97 以上，Content Substitution も 0.39 以上の ASR を示した．これらから商用 LLM は攻撃プロンプトが入力に含まれていれば指示に従うため，PDF ファイルを直接入力した際に攻撃が成立しなかった原因は商用 LLM の PDF 処理系が本文以外のオブジェクトを抽出していないことにあると考えられる．

© 2026 Information Processing Society of Japan

一方，Task Refusal は平文入力でも両商用モデルで 0 であった．これは，抑制的な指示に対する耐性がモデル自体の堅牢性に由来することを示しており，プロンプト種別によって防御が働く層が異なることが考えられる．

ローカル LLM の攻撃成功率は，悪意あるプロンプトの挿入 PDF オブジェクトによって変化しなかった．その理由は，PDF パーサがオブジェクトに関係なく LLM に理解できるプレーンテキストとして出力するためと考えられる．プロンプトの種類間で攻撃成功率に差が生じた要因として，各プロンプトが要求する動作の性質の違いが考えられる．「User Redirection」および「Content Substitution」は特定の URL や単語を「出力させる」能動的な指示であるのに対し，「Task Refusal」は正規のタスクを「実行させない」抑制的な指示である．LLM は明示的なタスク要求に従うよう強く最適化されているため，抑制的な指示には従いにくく攻撃成功率が低下したと考えられる．本研究で用いたプロンプトのうち「User Redirection」が最も多くの LLM に有効であり，RQ2 に対する一つの回答を与える．

また，漏洩率は最も高い Qwen3.5 でも最大 0.05 であった．これは，LLM が攻撃プロンプトの指示には従うものの被害者の目に触れる出力に痕跡がほとんど残らないことを示しており，提案手法が高い隠蔽性を有すると考えられる．

7.2 挿入プロンプトと PDF ファイル本文の文量比と攻撃成功率

Llama3.1-8B および Ministral3-8B では，本文の文量比が大きくなるにつれて攻撃成功率が低下する傾向が見られた．特に，文量比が 10 倍から 50 倍の間を境として攻撃成功率が急落し，50 倍以上ではほぼ 0 に近い値となった．これは，本文が長大になるほど挿入プロンプトの相対的な比重が低下する「希釈効果」によるものと考えられ，この傾向は RQ3 に対する一つの回答を与えるものである．

一方，Qwen3.5-9B は文量比が低い側で攻撃成功率がやや低く，10 倍付近で最大となる山型の傾向を示した．本文が短い場合は入力全体に占める攻撃プロンプトの割合が高く，正規の処理対象と区別されやすかった可能性が考えられる．また，Gemma4-12B は文量比に関わらず攻撃成功率がほぼ 1.00 で一定であった．以上のように文量比と攻撃成功率の関係に本実験の LLM で共通する傾向は確認されず，実験 1 の RQ2 と同様にモデル依存性が強いと考えられる．

7.3 関連研究との比較

本研究の提案手法と既存研究を比較し，有効性および限界を検討する．

提案手法は，Collu ら [1] が用いた極小白文字による本文挿入と比較して，ローカル LLM に対する攻撃成功率が高い結果となった．既存手法の攻撃成功率が低くなった原因を探るため前処理後のデータを確認したところ，前処理前には無かった余計なスペースが混入していたことが原因と考えられる．この空白混入が pypdf 固有の現象か確認するため，llama の既存手法に対し pdfminer と PyMuPDF でも同様に前処理を行った．結果を**表 9** に示す．pdfminer では pypdf と同様に空白の混入が確認された一方，PyMuPDF ではほとんど見られず，空白混入は pypdf 固有ではなく，程度の差はあるものの複数の前処理系で生じうる現象であることを確認した．

表 9 PDF パーサと既存手法の ASR

PDF パーサ	pypdf	pdfminer	PyMuPDF
Task Refusal	0.05	0.02	0.10
User Redirection	0.02	0.00	0.41
Content Substitution	0.64	0.83	0.75

既存手法は白文字のため肉眼での発見は困難だが，PDF 本文に記載されるため全選択時等に第三者に発覚し得る．一方，提案手法は PDF 本文に変化を与えないため隠蔽性が高いと考えられる．

評価の厳密性にも差異がある．Lian ら [5] は攻撃成否のみを確認しているが，本研究は各組み合わせ 100 回試行し ASR と漏洩率の 2 指標で定量評価した．

一方で提案手法の問題点は 2 点ある．1 点目は，各オブジェクトを正規に確認された場合に攻撃が第三者に知られてしまう点である．2 点目は，実行環境によっては PDF パーサが該当オブジェクトを処理せず攻撃が成立しない点である．裏を返せば，この前処理仕様こそが有効な防御策であることを意味しており，本文以外のオブジェクトを抽出しない，またはサニタイズするという PDF パーサ側の対策が，間接プロンプトインジェクション攻撃対策の指針となり得る．

7.4 本研究の限界

本研究の限界として，実験時の生成パラメータ設定が挙げられる．temperature および top-p をいずれも 0.9 に設定しており，実運用で多く採用されるより低い設定とは異なるため，観測された攻撃成功率のばらつきの一因となっている可能性がある．今後は複数の条件下での体系的な評価が必要である．

また，本研究の評価は pypdf による前処理を前提としている．実験 3 によりモデル要因とパーサ要因の切り分けは行ったが，pdfminer や PyMuPDF を用いた場合の攻撃成功率は検証していない．これらでも各オブジェクトから同様に攻撃プロンプトを抽出可能であることは確認しているが，パーサごとの定量的差異は今後の課題である．

さらに，本研究の評価範囲にも限界がある．本文は日本語の文学作品 1 点，正規タスクは 1 行要約のみを対象としており，英語文書や学術論文といった他ジャンル，情報抽

© 2026 Information Processing Society of Japan

出や RAG 等の他タスクでの検証は行っていない．また，対象とした商用 LLM は軽量ティアの 2 モデルに限られる．したがって「商用 LLM には有効でない」という結果を上位ティアを含む商用 LLM 全般に一般化することはできず，今後はより多様な文書・タスク・モデルでの評価が必要である．

8. 研究倫理

本研究で使用した PDF 本文は，著作権の保護期間が満了した夏目漱石の「吾輩は猫である」の一節を用いており，実在の個人情報や非公開情報は含んでいない．

本手法は PDF 編集ライブラリの標準機能のみで実現でき攻撃者の障壁が低く，近縁の白文字挿入も既に実際の事案として報告されている [6]．よって本研究は新規の攻撃能力を与えるものではなく，防御側が本経路を認識していない情報の非対称性の方が公表による追加リスクを上回ると考えられる．加えて本知見は本文以外のオブジェクトを除外またはサニタイズする防御策に直結する．実験 3 のとおりこの対策は商用 LLM で機能する一方，PDF パーサをローカルに実装する構成では本攻撃の影響を受け得るため，開発者に伝える意義がある．

開示範囲は防御に必要な情報と攻撃の実行に必要な情報の差分で線引きした．脅威モデル，挿入経路のオブジェクト種別，攻撃成功率および防御策は検査対象の決定に不可欠なため公開し，攻撃プロンプトの文言および注入スクリプトは転用可能かつ防御には不要なため公開しない．完全な非公開は再現性を損ない，完全な公開は PDF の流通範囲に鑑み妥当でないと判断した．

9. おわりに

本研究では,PDF ファイルの構造的特徴に着目し, 本文 (Contents) 以外のオブジェクトである埋め込みファイル,JavaScript,Actual Text,Alt に悪意あるプロンプトを挿入する間接プロンプトインジェクション攻撃を提案した. 実験の結果, 提案手法はローカル LLM に対しては挿入先オブジェクトの種類に関わらず高い攻撃成功率を示し (RQ1), 既存の白文字挿入手法よりも有効であった. 一方, 商用 LLM に対しては PDF ファイルを直接入力した場合には攻撃が成功しなかった. この原因を切り分けるため pypdf で抽出した平文を入力する追加実験を行った結果,User Redirection は 0.97 以上の ASR を示し, 攻撃が成立しない原因が商用 LLM の PDF 処理系が本文以外のオブジェクトを抽出しないことにあると確認した. この知見は, 本文以外のオブジェクトを抽出しないかサニタイズする前処理が有効な防御策となることを示唆している. またローカル LLM では「User Redirection」以外に普遍的に有効な攻撃は確認されず (RQ2), 文量比と攻撃成功率の関係 (RQ3) については本文が長大になるほど攻撃成功率が低下する希釈効果が見られたが, その傾向は LLM の種類に強く依存した. 今後は, より多様な文書・タスク・モデルおよび PDF パーサでの検証を行うとともに, 本研究で明らかにした攻撃を踏まえた防御手法の検討を進めたい.

参考文献

[1] Matteo Gioele Collu, Umberto Salviati, Roberto Confalonieri, Mauro Conti, and Giovanni Apruzzese. Misleading large language models used (or misused) in scientific peer-reviewing via hidden prompt-injection attacks. *ACM Transactions on AI Security and Privacy*, 2026.

[2] Tongcheng Geng, Zhiyuan Xu, Yubin Qu, and W. Eric Wong. Prompt injection attacks on large language models: A survey of attack methods, root causes, and defense strategies. *Computers, Materials & Continua*, 87(1):1–10, 2026.

[3] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, pages 79–90, Copenhagen, Denmark, 2023. Association for Computing Machinery.

[4] International Organization for Standardization. ISO 32000-2:2020 document management — portable document format — part 2: PDF 2.0. Geneva, Switzerland, 2020.

[5] Zhuotao Lian, Weiyu Wang, Qingkui Zeng, Toru Nakanishi, Teruaki Kitasuka, and Chunhua Su. Prompt-in-content attacks: Exploiting uploaded inputs to hijack llm behavior. In Jiageng Chen, Debiao He, and Wei Xie, editors, *Network and System Security*, pages 146–160, Singapore, 2026. Springer Nature Singapore.

[6] Nikkei Asia. "positive review only": Researchers hide AI prompts in papers. `https://asia.nikkei.com/business/technology/artificial-intelligence/positive-review-only-researchers-hide-ai-prompts-in-papers`, 2025. (Accessed: 2026-07-14).

[7] Nir Nissim, Aviad Cohen, Chanan Glezer, and Yuval Elovici. Detection of malicious PDF files and directions for enhancements: A state-of-the art survey. *Computers & Security*, 48:246–266, 2015.

[8] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. arXiv preprint arXiv:2211.09527, `https://arxiv.org/abs/2211.09527`, 2022.

[9] Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing*, 4(2):100211, 2024.

[10] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, Xiaolei Wang, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Yiwen Hu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models. *Frontiers of Computer Science*, 20(12):2012627, 2026.

[11] 読売新聞. 生成ＡＩ悪用しウイルス作成、有罪判決…ＩＴ知識なくとも「１か月ぐらいで簡単に作れた」. `https://www.yomiuri.co.jp/national/20241025-OYT1T50209/`, 2024. 読売新聞オンライン, (Accessed: 2026-07-14).

© 2026 Information Processing Society of Japan